

Relatório de Elaboração do Projeto

Sistema de Gestão de Encomendas de Restauro

Rose Maria Restauros

Curso: UFCD 10790 – Projeto de Programação

Autora: Yasmin Resende

Formador(a): Carlos Barata

Data: 19.06.2026

Sumário

1. Introdução
2. Descrição do Projeto
3. Levantamento de Requisitos
4. Análise de Sistemas
5. Design do Sistema
6. Implementação
7. Testes
8. Implantação
9. Conclusão
10. Anexos

1. Introdução

Contextualização

Este relatório documenta o processo de elaboração do projeto “Sistema de Gestão de Encomendas de Restauro”, uma aplicação desenvolvida em Python no âmbito da UFCD 10790 – Projeto de Programação. O projeto destina-se a apoiar a gestão diária de um pequeno negócio real de restauro e pintura de mobiliário, a Rose Maria Restauros, permitindo organizar clientes, serviços e encomendas num único sistema.

Objetivo do Relatório

O objetivo deste relatório é apresentar, de forma detalhada, todas as etapas de desenvolvimento do projeto, desde o levantamento de requisitos até à implementação e aos testes, descrevendo as decisões tomadas e a forma como a solução final responde aos requisitos definidos.

2. Descrição do Projeto

Visão Geral

O Sistema de Gestão de Encomendas de Restauro é uma aplicação de consola que automatiza e organiza as operações diárias de um ateliê de restauro: o registo de clientes, a consulta dos serviços oferecidos e a gestão completa das encomendas, desde a sua entrada até à entrega. Substitui um processo manual, disperso por cadernos e folhas soltas, por uma ferramenta centralizada, fiável e fácil de consultar.

Escopo

Incluído no projeto:

- Registo e consulta de clientes.
- Consulta dos serviços de restauro e respetivos preços.
- Criação e gestão de encomendas, com acompanhamento do estado de cada peça.
- Filtragem de encomendas e identificação de trabalhos em atraso.
- Relatórios de faturação e de distribuição por estado.
- Persistência dos dados em ficheiro local.

Não incluído no projeto:

- Processamento de pagamentos e faturação fiscal.
- Base de dados externa e interface gráfica.
- Acesso por parte dos clientes e funcionamento multiutilizador.
- Integração com plataformas ou serviços externos.

Objetivos

- Centralizar a gestão de clientes, serviços e encomendas.
- Acompanhar o estado e os prazos de cada encomenda.
- Proporcionar uma visão clara da faturação e da atividade do negócio.
- Reduzir erros e perdas de informação face ao processo manual.

3. Levantamento de Requisitos

Metodologia de recolha

Uma vez que o projeto serve um negócio real gerido por um familiar, os requisitos foram recolhidos através do conhecimento direto do funcionamento do ateliê e da observação do processo manual atualmente utilizado para gerir as encomendas. A partir dessa análise, identificaram-se as funcionalidades essenciais e as qualidades que o sistema deveria garantir.

Requisitos Funcionais

ID	Descrição
RF01	O sistema deve permitir registar um novo cliente com nome e contacto.
RF02	O sistema deve listar todos os clientes registados.
RF03	O sistema deve apresentar os serviços de restauro disponíveis e os respetivos preços base.
RF04	O sistema deve permitir criar uma encomenda associando cliente, serviço, descrição da peça, prazo e preço.
RF05	O sistema deve permitir atualizar o estado de uma encomenda (Recebida, Em progresso, Concluída, Entregue).
RF06	O sistema deve listar todas as encomendas registadas.
RF07	O sistema deve permitir filtrar as encomendas por estado e por cliente.
RF08	O sistema deve identificar as encomendas com prazo ultrapassado e ainda não entregues.
RF09	O sistema deve calcular e apresentar a faturação total das encomendas.
RF10	O sistema deve apresentar o número de encomendas em cada estado.
RF11	O sistema deve guardar e carregar automaticamente os dados a partir de um ficheiro local.

Requisitos Não Funcionais

ID	Descrição
RNF01	Usabilidade: menu de opções claro, navegável pelo teclado e utilizável sem formação prévia.
RNF02	Robustez: o sistema trata entradas inválidas sem terminar inesperadamente, apresentando mensagens de erro.
RNF03	Desempenho: cada operação responde em menos de um segundo em condições normais.
RNF04	Integridade dos dados: a informação é gravada no ficheiro após cada alteração, evitando perdas.
RNF05	Compatibilidade: funciona em qualquer sistema operativo com Python 3 (Windows, macOS, Linux).
RNF06	Manutenibilidade: código organizado em classes e funções identificadas e comentadas.
RNF07	Controlo de versões: código alojado num repositório Git/GitHub com histórico de alterações.

4. Análise de Sistemas

Modelação

O sistema assenta em três entidades principais: o Cliente, o Serviço e a Encomenda. Cada encomenda está associada a um cliente e a um serviço, e possui um estado que evolui ao longo do tempo (Recebida, Em progresso, Concluída, Entregue). Os clientes e os serviços são registados de forma independente e reutilizados na criação das encomendas.

Casos de Uso

Os principais casos de uso, todos realizados pelo gestor do ateliê (utilizador único), são:

- Registrar cliente: o gestor regista um novo cliente no sistema.
- Criar encomenda: o gestor regista uma nova encomenda, associando-a a um cliente e a um serviço.
- Atualizar estado: o gestor altera o estado de uma encomenda à medida que o trabalho avança.
- Consultar encomendas: o gestor lista, filtra e identifica encomendas em atraso.
- Consultar relatórios: o gestor consulta a faturação e a distribuição das encomendas por estado.

Análise de Riscos

Risco	Mitigação
Perda de dados por falha ou fecho inesperado da aplicação.	Gravação automática no ficheiro após cada alteração.
Introdução de dados inválidos pelo utilizador.	Validação das entradas com tratamento de exceções (try/except).
Dificuldade de utilização do sistema.	Menu de texto simples, numerado e intuitivo.
Datas mal formatadas a causar erros.	Validação do formato da data no momento do registo da encomenda.

5. Design do Sistema

Arquitetura do Sistema

Dada a dimensão do projeto e o âmbito da UFCD, optou-se por uma arquitetura simples: uma aplicação monolítica de consola, contida num único ficheiro (main.py) e organizada em quatro blocos lógicos distintos — classes, persistência, funções de operação e menu. Esta opção, em vez de uma arquitetura em camadas com base de dados, mantém o código acessível, fácil de explicar e adequado a um primeiro projeto de programação.

Design de Componentes

- Classes (Cliente, Serviço, Encomenda): representam os dados do sistema, aplicando Programação Orientada a Objetos.
- Funções de persistência (guardar_dados, ler_dados): tratam da gravação e leitura dos dados no ficheiro JSON.
- Funções de operação: cada funcionalidade (registar, criar, atualizar, listar, filtrar, relatar) corresponde a uma função própria.

- Menu: ciclo principal que apresenta as opções e encaminha cada escolha para a função correspondente.

Interface do Utilizador

A interface é em modo de texto (consola). Ao iniciar, a aplicação apresenta um menu numerado com todas as opções disponíveis. O utilizador escolhe uma opção digitando o número correspondente, e o sistema guia-o através de perguntas simples para concluir cada tarefa.

Persistência de Dados

Em vez de uma base de dados, o sistema guarda toda a informação num ficheiro de texto no formato JSON (dados.json), criado automaticamente na primeira utilização. Os objetos são convertidos em dicionários para serem gravados e reconstruídos em objetos quando a aplicação é reaberta. O ficheiro é atualizado após cada alteração, garantindo a integridade dos dados.

6. Implementação

Plano de Implementação

A implementação seguiu as fases definidas no cronograma:

- Fase 1: definição das classes que representam os dados.
- Fase 2: implementação da persistência em JSON.
- Fase 3: desenvolvimento das funções de cada operação.
- Fase 4: construção do menu e testes finais.

Tecnologias Utilizadas

- Linguagem de programação: Python 3.
- Bibliotecas: json e datetime (ambas incluídas na biblioteca padrão do Python).
- Ferramentas: Visual Studio Code (editor) e Git/GitHub (controlo de versões).

Estrutura de Código

Todo o código está no ficheiro main.py, organizado em quatro partes: (1) as classes, (2) as funções de persistência, (3) as funções de operação e (4) o menu principal. Esta separação clara facilita a leitura e a manutenção do programa.

Controlo de Versões

O controlo de versões foi assegurado com o Git, estando o código alojado num repositório público no GitHub, com commits regulares ao longo do desenvolvimento.

Repositório: github.com/YasminDreer/projeto-ufcd-10790

7. Testes

Plano de Testes

Dada a natureza do projeto, foram realizados testes funcionais manuais a cada funcionalidade, percorrendo todas as opções do menu. Para além do funcionamento normal, testaram-se também situações de erro (entradas inválidas), para confirmar a robustez do sistema, bem como a persistência dos dados (fechar e reabrir a aplicação).

Casos de Teste e Resultados

Funcionalidade testada	Resultado
Registar cliente	Sucesso
Criar encomenda	Sucesso
Atualizar estado da encomenda	Sucesso
Listar e filtrar encomendas	Sucesso
Identificar encomendas em atraso	Sucesso
Relatórios (faturação e por estado)	Sucesso
Entrada inválida (texto ou data incorreta)	Sucesso
Persistência (dados mantidos após reabrir)	Sucesso

Síntese dos Resultados

Todos os casos de teste foram concluídos com sucesso. A aplicação comportou-se conforme o esperado em todas as funcionalidades, manteve os dados entre utilizações e respondeu de forma controlada às entradas inválidas, sem terminar inesperadamente.

8. Implantação

Plano de Implantação

A aplicação destina-se a utilização local. Para a colocar em funcionamento basta ter o Python 3 instalado, obter o ficheiro main.py (a partir do repositório) e executá-lo com o comando “python main.py”. O ficheiro de dados é criado automaticamente, não sendo necessária qualquer configuração adicional.

Formação

Sendo uma aplicação simples e de menu intuitivo, a formação necessária é mínima. É disponibilizado um Manual do Utilizador que explica, passo a passo, como utilizar cada funcionalidade.

Suporte

O suporte é assegurado pela documentação fornecida (manual do utilizador e documentação técnica) e pela autora do projeto, responsável pela sua manutenção e eventuais correções.

9. Conclusão

Resumo dos Resultados

O Sistema de Gestão de Encomendas de Restauro foi desenvolvido de acordo com os requisitos especificados. Todas as funcionalidades previstas foram implementadas e testadas com sucesso, resultando numa ferramenta funcional, robusta e adequada à gestão diária do negócio.

Próximos Passos

- Possível evolução para uma interface gráfica, mais amigável para o utilizador.
- Substituição do ficheiro JSON por uma base de dados, caso o volume de dados cresça.
- Exportação de relatórios para outros formatos (por exemplo, PDF ou folha de cálculo).

Reflexão

O desenvolvimento deste projeto proporcionou uma experiência prática valiosa em programação. Permitiu aplicar conceitos fundamentais como a Programação Orientada a Objetos, a manipulação de ficheiros, o tratamento de exceções e o controlo de versões. Ao longo do trabalho, a resolução de pequenos problemas como o tratamento correto das datas e a comparação de nomes reforçou a importância de testar o programa de forma cuidada e de planear antes de programar.

10. Anexos

- Documento de Âmbito do Projeto.
- Grelha de Requisitos Funcionais e Não Funcionais.
- Cronograma das atividades (Gantt).
- Código-fonte da aplicação (disponível no repositório GitHub).
- Manual do Utilizador e Documentação Técnica.